Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester V)

AI and Application Development and Jera

Task 2

Roll No.:T050	Name: Aashu Yadav
Class:TYIT	Batch:1
Date of Assignment:29-7-2026	Date/Time of Submission:29-7-2026

1. Design PEAS description for an Online Food Delivery Agent and classify its environment.

code:-

```
# PEAS Description for an Online Food Delivery Agent
PEAS = {
    "Performance Measure": [
        "Fast delivery",
        "Correct order delivered",
        "Customer satisfaction",
        "Minimum delivery cost",
        "Timely delivery"
    ],
    "Environment": [
        "Customers",
        "Restaurants",
        "Roads",
        "Traffic",
        "Weather"
    ],
    "Actuators": [
        "Assign delivery person",
        "Send notifications",
        "Update order status",
        "Recommend route"
    ],
    "Sensors": [
        "GPS",
        "Customer order",
        "Traffic data",
        "Restaurant status",
```

```
        "Payment confirmation"
    ]
}
print("PEAS Description")
for key, value in PEAS.items():
    print(f"\n{key}:")
    for item in value:
        print("-", item)
print("\nEnvironment Classification")
print("Observable      : Partially Observable")
print("Agents          : Multi-Agent")
print("Deterministic   : Stochastic")
print("Episodic        : Sequential")
print("Dynamic         : Dynamic")
print("Discrete/Continuous : Continuous")
print("Aashu")
```

Output:-

```
PEAS Description

Performance Measure:
- Fast delivery
- Correct order delivered
- Customer satisfaction
- Minimum delivery cost
- Timely delivery

Environment:
- Customers
- Restaurants
- Roads
- Traffic
- Weather

Actuators:
- Assign delivery person
- Send notifications
- Update order status
- Recommend route

Sensors:
- GPS
- Customer order
...
Episodic        : Sequential
Dynamic         : Dynamic
Discrete/Continuous : Continuous
Aashu
```

2. Implement Breadth-First Search to find the shortest path in a simple maze.

Code:-

```
from collections import deque
maze = [
    [0, 1, 0, 0, 0],
```

```
    [0, 1, 0, 1, 0],
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0]
]
start = (0, 0)
goal = (4, 4)
queue = deque([(start, [start])])
visited = {start}
directions = [(1,0), (-1,0), (0,1), (0,-1)]
while queue:
    (r, c), path = queue.popleft()
    if (r, c) == goal:
        print("Shortest Path:", path)
        print("Path Length:", len(path) - 1)
        break
    for dr, dc in directions:
        nr, nc = r + dr, c + dc
        if (0 <= nr < len(maze) and
            0 <= nc < len(maze[0]) and
            maze[nr][nc] == 0 and
            (nr, nc) not in visited):
            visited.add((nr, nc))
            queue.append(((nr, nc), path + [(nr, nc)]))
print("Aashu")
```

Output:-

```
Shortest Path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (3, 2), (4, 2), (4, 3), (4, 4)]
Path Length: 8
Aashu
```

3. Apply Bayes' Rule to calculate the probability of rain given cloudy weather.

Code:-

```
# Bayes Theorem
P_rain = 0.30
P_cloudy_given_rain = 0.80
P_cloudy = 0.50
P_rain_given_cloudy = (P_cloudy_given_rain * P_rain) / P_cloudy
print("Probability of Rain given Cloudy =", round(P_rain_given_cloudy, 2))
print("Aashu")
```

Output:-

```
Probability of Rain given Cloudy = 0.48
Aashu
```

4. Solve the Missionaries and Cannibals problem using BFS or DFS.

Code:-

```
from collections import deque
def is_valid(state):
    m_left, c_left, boat = state
    m_right = 3 - m_left
```

```
    c_right = 3 - c_left
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False
    if (m_left > 0 and m_left < c_left):
        return False
    if (m_right > 0 and m_right < c_right):
        return False
    return True
def bfs():
    start = (3, 3, 1)
    goal = (0, 0, 0)
    queue = deque([(start, [])])
    visited = set()
    moves = [(1,0),(2,0),(0,1),(0,2),(1,1)]
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.add(state)
        m, c, boat = state
        for dm, dc in moves:
            if boat:
                new = (m-dm, c-dc, 0)
            else:
                new = (m+dm, c+dc, 1)
            if is_valid(new):
                queue.append((new, path+[state]))
solution = bfs()
print("Solution:")
for step in solution:
    print(step)
print("Aashu")
```

Output:-

```
Solution:
(3, 3, 1)
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 0, 0)
Aashu
```

5. Implement A* Search for a graph and compare its result with UCS.

Code:-

```
import heapq
graph = {
   'A': {'B':1,'C':4},
   'B': {'D':2,'E':5},
   'C': {'F':3},
   'D': {'G':5},
   'E': {'G':2},
   'F': {'G':2},
   'G': {}
}
heuristic = {
   'A':7,
   'B':6,
   'C':2,
   'D':4,
   'E':1,
   'F':1,
   'G':0
}
def astar(start, goal):
   pq = [(heuristic[start],0,start,[start])]
   visited = set()
   while pq:
      f,g,node,path = heapq.heappop(pq)
      if node == goal:
         return path,g
      if node in visited:
         continue
      visited.add(node)
      for neighbor,cost in graph[node].items():
         heapq.heappush(
            pq,
            (g+cost+heuristic[neighbor],
             g+cost,
             neighbor,
             path+[neighbor])
         )
def ucs(start, goal):
   pq = [(0,start,[start])]
   visited = set()
   while pq:
      cost,node,path = heapq.heappop(pq)
      if node == goal:
         return path,cost
      if node in visited:
         continue
      visited.add(node)
      for neighbor,c in graph[node].items():
         heapq.heappush(
```

```
                pq,
                (cost+c,
                 neighbor,
                 path+[neighbor])
            )
a_path,a_cost = astar('A','G')
u_path,u_cost = ucs('A','G')
print("A* Search")
print("Path:", a_path)
print("Cost:", a_cost)
print("\nUniform Cost Search")
print("Path:", u_path)
print("Cost:", u_cost)
print("Aashu")
```

Output:-

```
A* Search
Path: ['A', 'B', 'D', 'G']
Cost: 8

Uniform Cost Search
Path: ['A', 'B', 'D', 'G']
Cost: 8
Aashu
```